

Unit Conversion CLI Proposal

October 9, 2026

Contents

1. Summary
2. Background and problem
3. Prior research
4. Goal and scope
5. Milestone plan
6. Risks

References

Purpose. This document describes the plan to build a Python command-line tool that converts length, weight and temperature units using only the standard library, and seeks approval of the goal, the scope and the two milestones (M1 Conversion core, M2 Command-line interface).

1 Summary

The repository contains only one empty initial commit[D1]. This goal builds a tool that takes a value, a source unit and a target unit, as in `python convert.py 100 km mi`, and prints `62.13711922 mi`.

- **Method.** Length and weight are converted with a single table of factors relative to a base unit (m, kg); temperature is converted separately by formulas that go through kelvin. This is the distinction that Pint and GNU Units make[R4, R5].
- **Source values.** The conversion factors are the exact values from NIST and the 1959 international yard and pound agreement (1 in = 0.0254 m, 1 lb = 0.45359237 kg)[R1, R3].
- **Constraint.** No external packages; only the Python 3.11 standard library (`argparse`, `unittest`). There are two files: `convert.py` and a test file.
- **Plan.** M1 builds the conversion function and its tests; M2 builds command-line parsing and error handling.

2 Background and problem

The repository currently contains no code[D1]. The runtime is Python 3.11.16[D2]. The tool is small, but the key point of the design is that **the three kinds of unit are not converted the same way**.

Length and weight are converted by multiplication alone. 1 km is 1000 m, and 0 km is 0 m. Temperature scales have different zero points, so conversion needs an addition on top of the multiplication. 0 °C is not 0 °F but 32 °F[R1]. If a single factor table also handled temperature, this addition would be missing and the result would be wrong. Table 1 summarizes the difference.

Table 1. Conversion properties by kind of unit. “Linear” means a conversion of the form $y = a \cdot x$, and “affine” means $y = a \cdot x + b$. [R1, R4]

Kind	Supported units	Base unit	Conversion	Example
Length	mm, cm, m, km, in, ft, yd, mi	m	Linear	1 mi = 1609.344 m
Weight	mg, g, kg, t, oz, lb	kg	Linear	1 lb = 0.45359237 kg
Temperature	C, F, K	K	Affine	0 °C = 32 °F = 273.15 K

Invalid user input is also a problem. An unknown unit ($\text{km} \rightarrow \text{xx}$), units of different kinds ($\text{km} \rightarrow \text{kg}$), a non-numeric value and a temperature below absolute zero (0 K) must end in an error that states the reason, instead of a result.

3 Prior research

3.1 Existing tools

Well-known tools for unit conversion already exist. Table 2 compares whether this project can use them directly and what it can learn from them.

Table 2. Comparison of existing approaches. [R4, R5]

Approach	Description	Usable in this project?	Lesson
Pint (Python library)	A unit registry and quantity objects. Temperature is handled separately as a “non-multiplicative unit”, and temperature differences have separate delta units.	No. It is an external package, which violates the constraint.	Separate temperature as an offset conversion. Block multiplication mixed with temperature as an error.
GNU Units (command-line program)	A definition file with thousands of units. Absolute temperatures use function syntax such as <code>tempF(45)</code> ; temperature differences use ordinary units such as <code>degF</code> .	No. It is an external program that needs a separate installation.	Absolute temperatures and temperature differences must be distinguished. This scope covers absolute temperatures only.
Own implementation (factor table + temperature formulas)	Two dictionaries of unit $\rightarrow$ base unit factors, plus temperature conversion formulas.	Yes. A few dozen lines with the standard library alone are enough.	—

3.2 Conversion factors

The factors are not computed by hand; the defined values are used as they are. All values in Table 3 are exact by definition. The NIST table lists the pound rounded to 7 digits (0.4535924) [R2], so the pound uses the defined value from the 1959 international yard and pound agreement[R3]. The ounce is defined as 1/16 of a pound, so it is obtained by calculation.

Table 3. Exact conversion factors to be used. Metric prefixes of the metre and kilogram (mm, km, mg, etc.) are powers of 10 and are omitted. [R1, R2, R3]

Unit	To base unit	Source
inch (in)	0.0254 m	NIST SP 811 B.8, marked exact[R1]
foot (ft)	0.3048 m	NIST SP 811 B.8, marked exact[R1]
yard (yd)	0.9144 m	NIST SP 811 B.8, marked exact[R1]
mile (mi)	1609.344 m	NIST SP 811 B.8, marked exact[R1]
pound (lb)	0.45359237 kg	Defined by the 1959 international agreement[R3]
ounce (oz)	0.028349523125 kg	1 lb $\div$ 16
$^{\circ}\text{F} \rightarrow ^{\circ}\text{C}$	$(t - 32) \div 1.8$	NIST SP 811 B.8, constants exact[R1]
$^{\circ}\text{C} \rightarrow \text{K}$	$t + 273.15$	NIST SP 811 B.8[R1]

3.3 Command-line parsing

The standard library module `argparse` takes three positional arguments (value, source unit, target unit). Negative numbers need care. `argparse` may treat an argument starting with `-` as an option, but **it accepts an argument that looks like a negative number as a positional argument**[R6]. Before Python 3.14, however, this “looks like a negative number” check recognizes only simple forms such as `-40` and `-1.5`; exponent notation such as `-1e3` was added in 3.14[R6]. In 3.11, a negative number in exponent notation must be passed after `--`[R6].

3.4 Approach for this project

Figure 1 shows the conversion paths that combine the research above. Length and weight are multiplied into the base unit and then divided out of it; temperature is converted to kelvin and then converted again. In either case, n units need only n relations to the base unit, so there is no need to write down every unit pair (n^2 pairs).

(a) Length and weight (linear)



(b) Temperature (affine)



Figure 1. Two conversion paths. The bold border marks the intermediate value that every unit passes through. Length and weight (a) use only multiplication and division; temperature (b) uses a formula that includes addition. The numbers are actual example values.

4 Goal and scope

Goal: a single `convert.py` that uses only the standard library converts absolute values of length, weight and temperature with exact defined factors. The success criteria are listed in Table 4.

Table 4. Success criteria.

Criterion	Measurement	Target
Accuracy	Tests against the defined values in Table 3 and the reference points ($-40\text{ }^{\circ}\text{C} = -40\text{ }^{\circ}\text{F}$, $100\text{ }^{\circ}\text{C} = 212\text{ }^{\circ}\text{F}$, $0\text{ }^{\circ}\text{C} = 273.15\text{ K}$)	relative error $\leq 1\text{e-}12$
Round-trip consistency	$A \rightarrow B \rightarrow A$ conversion for every unit pair	relative error $\leq 1\text{e-}12$
Error handling	Unknown unit, kind mismatch, non-numeric value, below 0 K	4 / 4 give a message and exit code $\neq 0$
Dependencies	Check of the import list	standard library only

The following are out of scope this time. If they become necessary, they are handled as a new goal.

- **Temperature differences.** Delta conversion, such as converting “rose by $10\text{ }^{\circ}\text{C}$ ” to $^{\circ}\text{F}$ (which GNU Units and Pint handle separately[R4, R5]).
- **Compound units and other kinds.** km/h, area, volume, time, etc.
- **Configuration files and distribution.** User-defined units, a pip package, an interactive mode.

5 Milestone plan

As requested, there are two milestones. As Figure 2 shows, M2 only wraps the conversion function from M1, so it starts after M1 is finished.

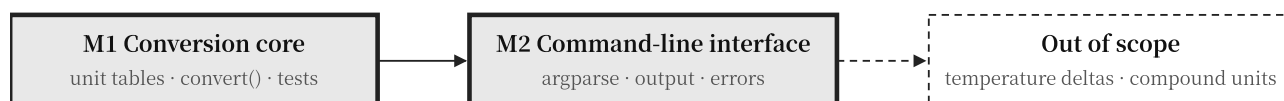


Figure 2. Milestone dependencies. The bold borders mark the two milestones of this goal; the dashed lines mark extensions that are out of scope this time.

Table 5. Contents and completion criteria by milestone.

Milestone	Contents	Completion criteria
M1 Conversion core	Length and weight factor tables, temperature formulas and a <code>convert(value, from, to)</code> function in <code>convert.py</code> . <code>unittest</code> tests in <code>test_convert.py</code> .	The accuracy and round-trip consistency tests in Table 4 pass. An unknown unit, a kind mismatch and a value below 0 K raise exceptions.
M2 Command-line interface	Takes a value and two units with <code>argparse</code> and prints the result (10 significant digits). Invalid input gives a one-line message and exit code 2.	<code>python convert.py 100 km mi</code> → <code>62.13711922 mi</code> . The tests for the four error types in Table 4 pass. Negative input (<code>-40 C F</code>) is handled.

6 Risks

The tool is small and so are the risks, but the four in Table 6 are prevented in advance by tests.

Table 6. Risks and responses.

Risk	Likelihood / impact	Response
Floating-point error	High / low. Example: $0.1 + 0.2 = 0.30000000000000004$	Output is rounded to 10 significant digits. Tests compare with a relative error of $1e-12$ instead of exact equality.
Negative number parsed as an option	Medium / medium. In 3.11, <code>-1e3</code> is not accepted as a positional argument[R6].	No option gets a name that looks like a negative number. A <code>-40</code> test is included; the error message and help text tell users to use <code>--</code> for negative numbers in exponent notation.
Ambiguous unit symbols	Low / low. The meaning of <code>C</code> versus <code>c</code> , and of <code>t</code> (tonne)	The design document sets the case rules and aliases. The unknown-unit error also shows the list of available units.
Wrong factor entered	Low / high. The result is silently wrong.	Factors are written exactly as the defined values in Table 3, and the tests check the defined values again independently.

References

- [R1] NIST. NIST Guide to the SI, Appendix B.8: Factors for Units Listed Alphabetically (SP 811). <https://www.nist.gov/pml/special-publication-811/nist-guide-si-appendix-b-conversion-factors/nist-guide-si-appendix-b8> (accessed 2026-10-09)
- [R2] NIST. NIST Guide to the SI, Appendix B.9: Factors for Units Listed by Kind of Quantity or Field of Science (SP 811). <https://www.nist.gov/pml/special-publication-811/nist-guide-si-appendix-b-conversion-factors/nist-guide-si-appendix-b9> (accessed 2026-10-09)
- [R3] Wikipedia. International yard and pound. https://en.wikipedia.org/wiki/International_yard_and_pound (secondary source; the original text of the 1959 agreement was not verified)
- [R4] Pint documentation. Temperature conversion. <https://pint.readthedocs.io/en/stable/user/nonmult.html>
- [R5] GNU Units. <https://www.gnu.org/software/units/>
- [R6] Python Software Foundation. argparse — Parser for command-line options, arguments and subcommands, “Arguments containing -”. <https://docs.python.org/3/library/argparse.html>
- [D1] Initial repository commit 254d145 “init” (no files), accessed 2026-10-09.
- [D2] Runtime check: `python --version` → Python 3.11.16, 2026-10-09.